IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 22 March 2025, accepted 23 April 2025, date of publication 28 April 2025, date of current version 5 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3564677

RESEARCH ARTICLE

HMSTNet: A Deep Learning Multimodal Approach for Personalized English Literature Recommendations

XIAOPENG WANG AND XUE LIANG
Langfang Health Vocational College, Langfang, Hebei 065000, China
Corresponding author: Xue Liang (19822021662@163.com)

ABSTRACT Recommendation systems serve as fundamental elements of contemporary technology by reshaping how users find content in all fields including educational applications. Academic English literature benefits strongly from recommendation systems because these systems enable customized learning opportunities to help students discover relevant content according to personal learning needs. English literature represents features challenging recommendation systems because it consists of extensive text content combined with multiple stylistic attributes and extensive metadata structures. The study presents a deep learning framework called HMSTNet to manage complex recommendation scenarios. Novel architecture integrates metadata datasets alongside stylistic information and BERT-powered text-based embeddings for understanding of content and user systemic preferences. Through its multi-branch structure Hybrid Multimodal Semantic Text Neural Network (HMSTNet) based on LSTM model to recognize difficult meta-relations between text components while utilizing user embeddings to maintain context awareness. This research serves to unite conventional recommendation algorithms with English literature reading requirements. The complex nature of literary data poses challenges to k-Nearest Neighbors (kNN) and Singular Value Decomposition (SVD) whereas conventional methods show limited success in both context understanding and accuracy. The model HMSTNet outperforms others with outstanding results reaching 95.23% accuracy. Multiple metrics demonstrate the efficiency of this model because they measure highest MBD with KNN model of 6.8m along with Theil's U-statistic of 0.04 and 90th percentile error is 8.41 to confirm minimized prediction errors and enhanced accuracy and fail-safe capability. Through its reliable framework HMSTNet both delivers personalized content recommendations and supports educational technology development which deepens student engagement with literature and promotes lifelong learning.

INDEX TERMS BERT embeddings, deep-learning, English literature, metadata, recommendation system, users personalization.

I. INTRODUCTION

Today's technology depends heavily on recommendation systems to help users find and interact with content in all types of platforms. Such systems use data to show users custom results which make navigation simpler and give user better experiences [1]. Recommendation systems developed advanced content-based filtering alongside their traditional approaches over many years. Content-based techniques examine item characteristics and user preferences at their core to thrive best in environments that heavily rely on metadata and text [2]. The value of recommendation systems stands out strongly in English literature. Too many books now make it hard for readers to easily find reading material they enjoy. Literature's wide array of writing approaches and subject material makes it hard for standard systems to match readers with suitable content [3]. By following individual reader preferences, the system suggests specific titles to meet their needs and introduce them to fresh entertainment options [4].

The associate editor coordinating the review of this manuscript and approving it for publication was Donato Impedovo.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Standard recommendation methods show major weaknesses when used in the detailed field of English literature research [5]. The most common recommendation method called Collaborative Filtering by using user input records to predict future behavior. In situations with limited user history or new releases the system demonstrates poor performance despite yielding good outcomes with substantial datasets [6]. The system needs user feedback to operate but fails to grasp content characteristics properly resulting in recommendation results that miss deep specific connections [7].

The special nature of English literature makes recommendation systems hard to navigate because of its complex metadata. Reading a piece of literature requires understanding its different types, thematic elements, and author's distinctive style as identified in research [8]. These standard systems cannot properly analyze the many types of metadata available to produce precise recommendations [9]. A text's natural language formation in literary descriptions creates major processing difficulties. Traditional recommendation systems find it hard to manage texts from literature because they process texts with complex words and hidden meanings plus different writing styles [10]. The system's problems show that advanced methods must manage metadata analysis along with natural language processing tasks to give superior recommendations for English literature [11] and graph-based structures can further enhance the predictive accuracy by incorporating more powerful users' interaction [12].

These advanced systems need to recognize user preferences better because customers want custom recommendations that match their tastes [13]. Regular systems lack accuracy when making personal recommendations, so they perform badly with literary materials whose content ranges widely. Current recommendation technologies need to incorporate deep learning methods and semantic text understanding methods for better results [14], although for missing rating handled based on generative learning [15] in literary recommendations. These missing elements in the system make it hard to recommend literary works accurately and lower recommendation quality. Specific user behavior and preference analysis using user embedding techniques has not been fully explored in recommendations but has held promise since [16]. Using this approach allows us to customize recommendations based on what individuals like to read, coupled with multimodal approaches to recommend literature which is aligned to the preferences of individual user [17]. The research aims to overcome previous system constraints by examining how innovative technologies enhance literature recommendation services.

Our main aim of this research is to creates a content recommendation system for English literature. that merges different types of data to make better literary recommendations by processing metadata information alongside stylistic markers and contextual inputs. The system will generate better literary recommendations because it uses multiple data types to assess writings correctly. The study develops personalization abilities by teaching the system to use user embeddings which let it generate recommendations based on a user's personal taste and reading habits. Furthermore, the study strives to fill literature recommendation technology gaps for English works while building a stronger recommendation system for readers.

The key contributions of the study include:

- Integration of BERT-based embeddings with text embeddings, stylistic and metadata features for deeper contextual understanding of literary content.
- Context-aware representation learning is achieved using pre-trained BERT embeddings and LSTM networks for improved semantic understanding.
- Incorporation of user embeddings to enhance personalization and align recommendations with individual preferences.
- Comprehensive evaluation of the proposed system using real-world datasets to validate its effectiveness and practical applicability.

In the remainder of this paper, as defined in figure 1, Section II provides a review of related work and existing approaches in the field of recommendation systems, particularly focusing on their application to English literature. In Section III, the methodology is detailed, covering the data preprocessing steps, feature extraction techniques, and the architecture of the proposed deep learning model. Section IV presents the experimental results and the evaluation of the model's performance, demonstrating its effectiveness. Finally, Section V concludes the study by summarizing the key findings and suggesting potential directions for future research.

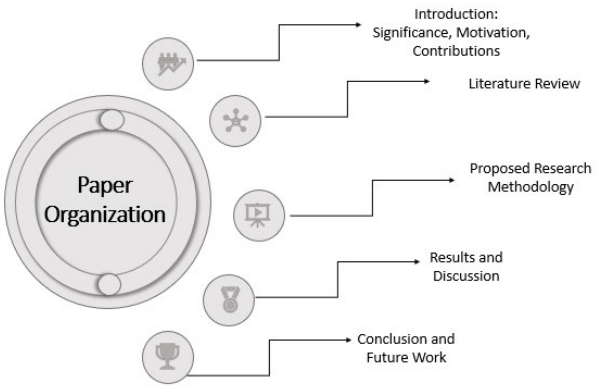


FIGURE 1. Steps showing paper structure.

II. RELATED WORK

This section examines different ways to combine Language Models and recommendation engines to develop more precise suggestions and tailor recommendations to specific users while generating explanatory reports. These methods employ diverse methods including hybrid prompting for products, sequential learning methods and adding graph features based

on global and users content information for recommendation systems yet face multiple hurdles in practice [18].

Researchers [19] suggest LLaRA to pair traditional recommendation technologies with big language models to process user actions and wide world knowledge. The system combines traditional models that produce item IDs with text-based item features through a projector to create hybrid prompts. A training plan starts with text prompts before mixing them with item-specific behavioral information to help the model blend both types of knowledge. The tests confirm that LLaRA boosts its ability to provide accurate recommendations. The model confronts difficulties because it requires pretrained LLMs plus demands excessively complex setups with sensitive data quality needs. The research work introduces Rec-former which presents a special framework for sequential recommendations that uses language representations to estimate the following item in someone's item viewing order. Recformer uses a bidirectional Transformer design with unique embeddings to match sequential recommendations needs like Long former [20]. The model combines training approaches to learn item sequences by combining text understanding skills with behavioral recommendation methods. The research evaluated Recformer using six datasets where the model achieved high prediction precision in situations with few resources and fresh data. The technique needs to rely on item key-value attributes from descriptions which cannot fully manage complex recommendation tasks. Deep learning has demonstrated significant potential in enhancing personalized learning experiences across various domains by proposing an optimal approach for predicting cognitive performance in educational settings, highlighting the effectiveness of AI in recommending and tailoring learning resources to individual needs [21]. Similarly, Another study, aligned for the need for intelligent recommendation systems that personalize content delivery based on learners' cognitive and behavioral patterns, [22] explored factors influencing deep learning adoption among EFL students, emphasizing the role of adaptive technologies in improving engagement and comprehension based on learning content. A research team [23] developed P5 which offers a text-to-text system for recommendation tasks while remaining adaptable and integrated. P5 converts multiple types of recommendation data from user interactions, user profiles, product information, and reviews into shared textual form. The model learns hidden patterns in text data and recommends products by sharing language modeling tasks during training. P5 works as a basis for different recommendation models and helps integrate new input types simply while producing instructions for product suggestions. The approach takes shallow recommendation models to deep scalable models while making progress toward a single recommendation system for all needs. P5 tailors prompt each user while managing zero-shot and few-shot predictions so training becomes more efficient. The analysis shows that the model achieves solid performance measures across different testing datasets. Despite its ability to process text through natural language processing LC-Rec struggles when making effective recommendations for tasks dependent on non-textual input data [2].

The paper presents LC-Rec as a unique recommendation model which merges language and collaborative semantics into large language models (LLMs) for solving recommender systems challenges. The system produces item recommendations straight from all available items instead of using preselected candidates. The model makes two key contributions: The paper uses a learning algorithm and uniform semantic mapping to create understandable item identity matches then adds specialized training steps to attach collaborative information to LLMs [24]. The system uses custom tasks to merge language and collaborative properties to enhance recommendations for LC-Rec performance in recommendations. Detailed tests prove that LC-Rec produces better results than numerous benchmark alternatives regardless of their LLM or traditional basis. The system demands more processing power because it requires fine-tuning operations which push up workload in limited computational settings.

The researchers introduce LLMRec which uses three unique graph augmentation methods built on language models to better suggest items. The strategies use online content from Netflix and MovieLens to enhance user-item connections while enriching item nodes and studying user behavior through written input. Using these methods LLMRec solves the issues of scant implicit feedback and poor-quality side information in recommendation systems. The framework uses denoised data robustification to improve augmented data quality through both noisy implicit feedback removal and MAE feature enhancement [25]. The system improves data quality by ensuring an accurate enhancement result. Research is presented that demonstrates how LLMRec optimizes models better than other systems. The results demonstrate that LLMRec generates better recommendations than current standard techniques based on test dataset analysis. The model's use of extra information leads to more complex operations and increased computing expenses. The paper [26] introduces KAR, an LLM based model to acquire two types of external knowledge: The system links knowledge about user preferences through reasoning with knowledge about specific items from facts. The framework uses factorization prompting to find genuine user preference insights. The hybrid-expert adaptor converts generated reasoning and facts into adapted vectors that match the recommendation task requirements. These vectors help make any recommendation algorithm perform better. LLMs provide knowledge to the recommendation system through a preprocessing stage of storage. Through many tests KAR demonstrates better results than other established platforms while working well with different recommendation techniques. Another study deployed KAR on Huawei news and music outlets to see how well it worked in online experiments. News recommendations

gained 7% extra success while music picks delivered a 1.7% performance increase. Efficient knowledge storage needs exist because this method depends on existing stored data.

The paper [27] presents HSS a hierarchical seq2seq framework with self-cleaning technology that generates individualized product suggestions and natural language descriptions. The model overcomes the problem of irrelevant user review sentences through feature-aware attention processes that identify suitable sentences based on item characteristics to create more personalized explanations. The system shows better recommendation performance and explanation updates for several online marketplaces through BLEU, ROUGE, and feature word measurements [28]. User reviews serve as their input yet present constraints such as insufficient or generalized details making personalized recommendations challenging in sparse review scenarios. Because of its self-cleaning process the model could perform poorly on items with vague characteristics and would need regular updates to match present user changes.

Another research based on RecMind an autonomous recommender that uses an LLM to recommend items based on external knowledge while planning tool usage in zero-shot mode [29]. The Self-Inspiring (SI) algorithm makes the LLM plan better by referring to previous searches alongside its access to historical purchasing data. The experiments prove that RecMind produces better recommendation results than present zero-shot and few-shot deep learning models while matching the effectiveness of P5 at its best configuration. RecMind uses the Self-Inspiring technique which merges multiple reasoning routes into its system to deliver better outcomes than CoT and ToT methods. The approach has a main weakness because zero-shot recommendations need more training for datasets and applications that are hard to manage. Guidance by SI helps with reasoning but its benefits are reduced in advanced recommendation work and situations with minimal or incomplete stored data [30].

Here is a table 1 summarize the details of the research papers mentioned:

III. PROBLEM STATEMENT

Basic recommendation engines that use collaborative filtering run into problems processing the intricate range of English literary works. The current systems find it difficult to work with deep metadata and contextual details of English literature works making their suggestions basic and not customized to users. The current systems do not properly merge user preference data into their recommendations which hampers their ability to customize output. The advanced literature-recommendation system needs to combine various data types including author metadata, language patterns, and reading behavior to produce better tailored English literature suggestions. Let R represent the recommendation system, where the output is a set of recommended items R_{rec} for a given user u. Traditional recommendation systems, such as collaborative filtering, rely on user-item interactions X_{ui} and produce recommendations based on historical feedback. However, these systems face the following challenges rely on $R_{collab} = f(X_{ui})$ to struggle for accurately modeling the intrinsic qualities of items due to their dependence on user feedback. This leads to suboptimal recommendations, especially in scenarios where user interaction history is sparse, i.e., *Sparse* (X_{ui})

Let M_{meta} represent the metadata of a literary work (such as genre, theme, and style), and T_{text} represent the textual description of the item, which includes stylistic features and vocabulary. Traditional systems fail to incorporate these rich metadata and textual elements into the recommendation process as $R_{meta} = f(M_{meta})$ *and* $R_{text} = f(T_{text})$ are inadequately integrated into R_{collab}. The challenge is further compounded by the lack of contextual understanding. A deeper understanding of the literary content is needed, which can be represented as $C_{text} = g(T_{text})$. Where g is a function that captures the complex meaning, structure, and nuances within the text. Traditional systems do not effectively model C_{text}.

Let U_{emb} represent the user embeddings that capture user preferences and behaviors. The current recommendation systems do not effectively utilize U_{emb}, resulting in a mismatch between user interests and the system's suggestions: $R_{emb}f(U_{emb})$ Thus, the objective of this research is to develop a content-based recommendation system R_{new} that integrates multiple sources of information: $R_{new} = f(M_{meta}, T_{text}, C_{text}, U_{emb})$. Where the system combines metadata M_{meta}, textual information T_{text}, contextual understanding C_{text}, and user embeddings U_{emb} to provide a more accurate, personalized, and context-aware recommendation for English literature.

IV. RESEARCH METHODOLOGY

The Proposed Methodology, defined in figure 2 explains how we built recommendation system using a structured development process for English literature applications. The process starts with Data Preprocessing which explains how to prepare metadata and text to make them compatible before merging.

Feature Engineering selects valuable writing style elements and contextual information from literary texts using metadata embedding and BERT-based text methods. System Design presents the deep learning structure that takes multiple data sources such as metadata, written text, and user preference records. Algorithm 2 delivers precise literature recommendation results by combining several distinct types of deep learning input.

A. DATA PREPROCESSING

The data preprocessing process begins with reading essential English Literature recommendations data from a CSV file. The dataset consists of columns that show genre, author, page count, summary, and user assignment values. We start by putting the dataset into a DataFrame through equation 1.

$$D = dataframe.read(CLGD.csv) \tag{1}$$

TABLE 1. Analysis of existing studies.

Model	Technique	Novelty	Improvement	Limitation
LLaRA [19]	Hybrid prompting method combining ID-based item embeddings and textual features.	Combines traditional recommenders' user behavior capture and LLMs' world knowledge encoding.	Improved recommendation performance with hybrid prompting and curriculum learning.	Rely on pre-trained LLMs; alignment and training complexity; sensitive to data quality.
Recformer [20]	Bi-directional Transformer architecture for sequential recommendations.	Uses language representations to predict the next item in a sequence.	Effective in low-resource and cold-start settings; excels in sequential recommendation tasks.	Relies on key-value item attributes described by text, which may not capture the complexity of all scenarios.
P5 [23]	Text-to-text paradigm for recommendation tasks, converting data into a natural language format.	Unifies multiple data types (user-item interactions, metadata, etc.) into a common natural language format for deeper semantics.	Supports zero-shot and few-shot predictions, reducing fine-tuning needs.	May perform poorly with non-textual data in recommendation tasks.
LC-Rec [24]	Integration of language and collaborative semantics for recommendations.	Generates recommendations directly from the entire item set, eliminating the need for candidate items.	Outperforms traditional and LLM-based recommenders in terms of performance.	Increased computational complexity due to reliance on fine-tuning, especially in resource-constrained environments.
LLMRec [25]	Three LLM-based graph augmentation strategies for enhancing interaction graphs.	Augments user-item interaction graphs and item node attributes using natural language.	Improves recommendation performance by addressing sparse feedback and low-quality side information.	Additional complexity and computational costs due to reliance on augmented data.
KAR [26]	Factorization prompting to acquire reasoning and factual knowledge on users and items.	Uses hybrid-expert adaptor to transform reasoning and factual knowledge into augmented vectors for recommendations.	Outperforms state-of-the-art baselines; show improvements in real-world platforms.	Reliance on pre-stored knowledge may require additional storage and management.
HSS [27]	Hierarchical sequence-to-sequence model with auto-denoising for personalized recommendations.	Selects relevant sentences for explanations using feature-aware attention mechanism in user reviews.	Improves recommendation accuracy and explanation quality across e-commerce domains.	Relies on potentially insufficient user reviews; struggles with poorly defined item features and adapting to changing user preferences.
RecMind [29]	Self-Inspiring algorithm for zero-shot personalized recommendations using LLMs.	Employs "Self-Inspiring" to enhance planning by considering historical states.	Outperforms exist zero-shot baselines and achieve performance comparable to fully trained models like P5.	Limited by complexity of certain domains or datasets, effectiveness may decrease in complex tasks with sparse historical data.

The program loads information from CLGD.csv to work with. The system prepares raw data to become workable through the data arrangement. The system stores book categories into a series.

$$g = [g_1, g_2, \ldots, g_n]\,(list\ of\ genres) \tag{2}$$

where each g_i corresponds to a specific genre associated with a book. Since the genres column contain categorical data, one-hot encoding is applied to this column to convert the genres into a binary format. This is done by applying the oneHotEncoder function, which transforms each genre into a separate binary feature indicating the presence or absence of that genre for each book. One-hot encoding is crucial in content-based recommendation systems because the relationships between books and genres are allowed to be understood in a structured way, making it easier for the model to process categorical features during training, represented by equation 3.

$$p = [p_1, p_2, p_3, \ldots, p_n]\,(list\ of\ pagenumbers) \tag{3}$$

which are numerical data, are next normalized using a StandardScaler. Normalization ensures that the page numbers are scaled to a standard range, typically with a mean of 0 and a standard deviation of 1. It standardizes the values of the pages to make them more comparable to other features such as user ratings. This is important because it ensures that features with larger numerical ranges are prevented from dominating the learning process, improving the performance of the recommendation model.

Text data preprocessing is then managed by cleaning the text data. The descriptions of the books are tokenized using the Tokenizer function, which fits a tokenizer to

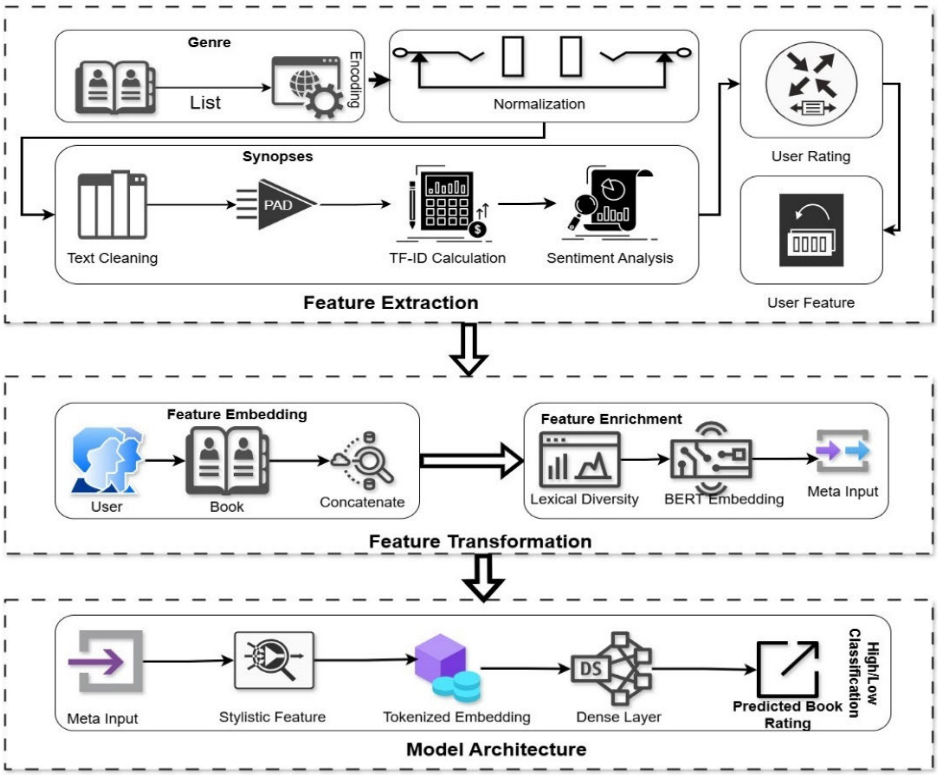


FIGURE 2. Steps showing framework of proposed methodology.

the descriptions, and text-to-sequence converts the text into sequences of integers. This step converts the text into a format that can be processed by machine learning models. After tokenization, the sequences of tokens are padded using the equation (4).

$$T_{padded} = pad_seq(T_{tokenized}, maxlen = L_{max}, padding = 'post', truc = 'post') \quad (4)$$

This makes sure every text portion comes with matching length. Neural networks depend on padding and truncating inputs because they need equal sequence lengths and must shorten longer sequences to their maximum size. We extract the Term Frequency-Inverse Document Frequency features from each book summary. Neural networks use this system to measure how much specific words matter when reviewing an entire collection. The TF-IDF system evaluates term importance in each document through this equation, which is multiplied by the log of the entire corpus size divided by the number of documents.

$$TF - IDF_{ij} = \left(\frac{count\ of\ term\ j\ in\ doc\ i}{Total\ terms\ in\ doc\ i}\right) \times \log\left(\frac{N}{DF_j}\right) \quad (5)$$

The calculation uses document total count N to determine term value against document frequency DF_j

The recommendation system uses TF-IDF values to understand book content better. The team employs a sentiment analysis pipeline trained for the job to analyze the book synopses. The analysis delivers sentimental results that show how texts feel emotionally to readers. Sentiment integration in the model helps us personalize book suggestions by recommending books that match a reader's emotional preferences. We normalize user ratings by using Min-Max normalization. The normalization method guarantees ratings always fall within the range of 0 to 1. Normalizing user ratings before model input prevents small rating variations from controlling recommendation results. The system takes user preferences specifically their favorite book genres. User IDs link separate entries holding data about preferred genres as fav-genre. The system learns what each individual user likes to generate better customized recommendations.

Deep learning model generates dense feature arrays from books and users to show their unique patterns. The model combines user and book representations into a single output vector by using equation 6.

$$Dataset_{output} = Concatenate(user.Embedding, book.Embedding) \quad (6)$$

The input shows how the system has trained each user embedding and book embedding. By allowing the embeddings to update during training the model captures new user choices and book traits as it progresses. Preprocessing methods prepare the data for deep learning by removing errors and normalizing values before transforming

it into recommendation system training. Processing text and metadata data before recommendation helps systems recognize and understand user-book relationships effectively. The personalized recommendation system improves user English Literature experience through deep learning-powered customization according to each user's specific preferences.

Deep learning-based recommendation systems need processed data as their starting point before they can analyze the dataset. The model requires us to modify categorical inputs to create processed data sets. The process starts by converting genre and author data into One-Hot Encoding format. Through One-Hot Encoding we transform categorical features into binary-coded vectors that use 0s and 1s to depict categories. This conversion technique lets the neural system grasp categorical patterns in genre and author relationships faster than the algorithm could alone. The model gains genre and author-specific understanding through its capacity to manage binary versions of categorical data. In equation 7 the phase ''One-Hot Encoding'' defines how to transform categorical data.

$$OneHot\ (x_i) = \begin{cases} 1 & if\ x_i = category_k \\ 0 & otherwise \end{cases} \tag{7}$$

where x_i represents a category in the data, and $category_k$ is the specific category that x_i is being compared to. This step transforms categorical features such as genres and authors into binary vectors suitable for the model to process.

B. FEATURE ENGINEERING

Transforming unprocessed text into useful linguistic features helps the model better recognize English literature because it finds important textual details during content prediction. During feature engineering's Feature Embedding phase we turn book and user data into embeddings to prepare them for further analysis. The model absorbs user and book traits in separate embeddings that merge to create an undistorted summary. The model merges user preference and book attribute embeddings to build a better feature space which boosts predictive accuracy. During Feature Enrichment the process uses these representations as input to add new data about their context and meaning. We evaluate for word variety in texts and add BERT embedding results to the final representation. The measure of lexical variety enriches the features through linguistic style representation [31]. BERT embeddings from a deep learning model help models understand sentences at detailed text context levels. After enhancement, the model receives input enriched with contextual and detailed information for ongoing processing. The model calculates Lexical Diversity with its measurements from book descriptions. This tool converts each book's description into distinct word units. Vocabulary range statistics in text show how diverse and refined a book's language appears to readers. When we know these stats we better understand the range of subject matter and writing approach authors use in their books. The Tokenizer manipulates text descriptions before sending them to model for processing. The model analyzes patterns in sentences to stop words and phrases that most effectively show what the work is about. To calculate Lexical Diversity the model uses equation 8.

$$LexDiv = \frac{T}{V} \tag{8}$$

Lexical Diversity is computed by dividing total word count T by the number of distinct words V found in the book description text.

The program uses BERT Tokenizer technology to extract more precise meaning from book descriptions while relying on Bert pre-trained models for analysis. As a transformer-based model BERT processes text from both directions to extract context information from sentences and paragraphs more effectively. To process a passage the system examines words that appear before and after a targeted word to grasp its meaning. BERT Tokenizer transforms book descriptions into manageable word pieces for BERT model processing. The Bert Model allows us to transform raw text input into useful numeric contextual embeddings using the BERT Pre-trained Model. To obtain embeddings features from BERT output the model uses Equation 9 to process each word in the input text. The label w_i defines its position within the description and $EBERT$ represents its embedding values.

$$E_{BERT} = BERT_{Pretrained}(w_i) \tag{9}$$

where w_i is the ith token in the description, and $EBERT$ is the contextual embedding vector for that token. Once the textual features are processed, the model moves to the next phase, where it builds a complex architecture for making predictions. The Meta Input Layer is constructed by combining several processed features: the one-hot encoded genre and author information, along with the tokenized text that has been enriched by the Lexical Diversity measure and BERT embeddings. The Meta Input Layer can be represented as in equation 10.

$$\begin{aligned} MetaInput = [&OneHot\left(G_{genre}\right) \\ &+ OneHot\left(A_{author}\right) + LexDiv + E_{BERT}] \end{aligned} \tag{10}$$

where G_{genre} and A_{author} are the genre and author features after One-Hot Encoding, and E_{BERT} is the embedding for the book description. The Lexical Diversity feature joins with other components to make a complete model input. A Dense Layer is placed directly after the meta-input. In a Dense Layer all nodes connect directly to every node from the earlier layer [32]. The transformation operations of the Dense Layer appear in equation 11.

$$h_{dense} = W_{dense}.MetaInput + b_{dense} \tag{11}$$

where W_{dense} is the weight matrix, b_{dense} is the bias vector, [33] and h_{dense} is the output of the Dense Layer. The activation function σ is applied to this output to introduce non-linearity.

C. PROPOSED MODEL ARCHITECTURE

The Stylistic Dense Layer is applied similarly to the input features, capturing stylistic elements of the text. Equation 12 shows the stylistic transformation.

$$h_{stylistic} = W_{stylistic}.MetaInput + b_{stylistic} \tag{12}$$

where $W_{stylistic}$ is the weight matrix for the stylistic features, and $b_{stylistic}$ is the bias vector. The output is passed through an activation function σ. The Embedding Layer is then applied to the tokenized text from the book descriptions. The Embedding Layer learns continuous vector representations of words, and the transformation for each token is represented by equation 13.

$$T_{embedding} = W_{embedding}.T_{tokens} + b_{embedding} \tag{13}$$

where T_{tokens} are the tokenized inputs, $W_{embedding}$ is the embedding weight matrix, and $T_{embedding}$ is the resulting vector for each word. Afterward, the LSTM (Long Short-Term Memory) layer is applied to the embedded tokens. LSTM is a type of RNN that can capture long-range dependencies within sequential data. The equation 14 for an LSTM layer output.

$$h_t = LSTM\left(T_{embeddings}\right)t \tag{14}$$

where h_t is the hidden state at time step t, and $T_{embeddings}$ is the sequence of word embeddings passed into the LSTM. This allows the model to capture patterns over the entire book description. Once all the features have been processed through the layers, the model makes its final prediction using a Dense Output Layer [34]. This output layer computes the predicted rating for each book, represented by equation 15.

$$r_{pred} = W_{output}.DenseOutput + b_{output} \tag{15}$$

where W_{output} and b_{output} are the weight matrix and bias vector for the output layer, and r_{pred} is the predicted rating. The algorithm classifies the predicted ratings as either high or low based on a threshold of 0.5. The classification is represented as shown in equation 16.

$$y_{class} = \begin{cases} high\ rating & if\ r_{pred} > 0.5 \\ low\ rating & if\ r_{pred} \leq 0.5 \end{cases} \tag{16}$$

In this classification, y_{class} represents the final recommendation as either high or low based on the predicted rating r_{pred}.

The algorithm can generate highly personalized book recommendations based on a comprehensive understanding of the book content, writing style, genre, and author preferences. The significance of each technique lies in its ability to process different types of data effectively, from categorical variables to complex textual content. Together, these techniques enable the algorithm to learn from a rich dataset and provide users with relevant, high-quality recommendations for English literature books that align with their tastes and preferences. The details regarding the implementation of the proposed model are given in algorithm 1.

Algorithm 1 Multimodal Deep Learning for Recommendation of English Literature Content

Input: Enriched pre-processed Comprehensive Literary Greats Dataset

Output: Predicted ratings based recommended books of English Literature

$E_g = OneHotEncod(G_{split(data['genres'])})$

$E_a = OneHotEncoder(A_{split(data['author'])})$

For ***Every*** d ***in*** *dataset* D

  $L_{div} = LexDiv(Tokenizer(data\,[description]))$

  $T_{Bert} = BertTokenizer(PreTrained)$

  $Model_Bert = TFBertModel(PreTrained)$

End For

For ***Each*** *Layer* ***in*** *Model*

  $Meta_{input} = input\ (shape\ (g_{enco} + a_{enco} + Pnorm))$

  $Meta_{dense} = Dense(n, activation)(M_{input})$

  $S_{stylistic} = Dense(u, acitvation)(input\ (shape, f))$

  $T_{embedding} = Embed(Idim = v, Odim = d)(input(shape, t))$

  $T_{lstm} = LSTM\ (h)\ (Tembedding)$

End For

$M = model(input\left[X_{metadata}, X_{stylistic}, X_{text}\right], Y_{output}$

$$\hat{r} = rating - predictedwhere, \hat{y} = \begin{cases} high\ rating & if\ r^{\wedge} > 0.5 \\ low\ rating & if\ r^{\wedge} \leq 0.5 \end{cases}$$

V. EXPERIMENTAL SETUP

A. DATASET

The Literary Greats Dataset is a vast treasure trove which contains more than 50 thousand books of world literature, the best samples of world literature, poetry, and authoring. The dataset has a total size of 638MB, as structured information that allows for analyzing literary works from a comprehensive perspective. This reflects the diversity of the dataset in terms of the genres, languages, and the spans of publication of the works. In addition, book ratings, awards, no. of votes etc. are included as metadata to evaluate the popularity and reception trends of the book. This data set provides all the following information pertaining to each book which can be useful to understand the reception and appraisal of books, as attributes are covered in table 2. Other characteristics are publisher, the number of pages, the book published and the first publication date, and the theme which is the setting of the stories contained in the books to get a better understanding of the book in consideration and its attractions to the readers. This dataset also includes other useful data including price of the book and cover image also

TABLE 2. Attributes of dataset.

Sr. No	Attributes	Sr. No	Attributes
1	Title	9	Characters
2	Author	10	Award
3	Average Rating	11	Book Format
4	Description	12	Best Books Ever Score
5	Language	13	Number of Votes received
6	ISBN	14	publisher
7	Genres	15	number of pages
8	book's publication date	16	first publication dates

enhances readers' experience. Although the dataset is rich, the cold start problem needs to be managed because there are no prior interactions between new books and new users. To address this issue, filter content using book attributes such as genre, description, and author details. Furthermore, different hybrid techniques of collaborative filtering are introduced to improve the model's capability of prediction for the newly introduced books. Preprocessing techniques, such as managing the missing values, standardizing the numerical attributes, and encoding the categorical variables were applied to guarantee data quality. Specifically, these steps help to improve the usability of the dataset and facilitate better performance of predictive models from it. Content based filtering is used for new books, where the metadata (e.g., genre, author, description, and awards) are used to find similarity with other books. The model will use Natural Language Processing (NLP) techniques to analyze textual descriptions, once the literature is uploaded, to identify how new books relate to those that are already in the system and can recommend them in the absence of historical interaction data. The model also offers recommendations that depend on demographics as well as the demographic information of the user in case it is a new user, analyzing user attributes and suggesting an initial set of suggestions from popular categories. Furthermore, the implicit preference approach is utilized, and the system suggests books on the trended, best-rated and the award-winning books until sufficient user interaction data is collected. The Cold start problems have been mitigated by a hybrid approach that combines some collaborative filtering and content-based techniques. For initializing recommendations, personalization improves as more data becomes available, and preference learning is refined through matrix factorization and hybrid deep learning-based embeddings when a user interacts with such initial recommendations.

To this research, where deep learning model is used for recommending English literature content, this research' s dataset is crucial. In its core it provides a versatile spectrum of tools that can be applied for developing customized recommendation solutions. Due to the great number of books, it allows adopting certain approaches to analyze user preferences in more detail. The structure of the dataset provides the opportunity to examine at least one two-variable relationship including, for example, genres, ratings, or even settings of books, which determine whether a reader will pick up a particular book or not. The features include rating and genre of books, and adopting a hybrid deep learning model it is possible to process and analyze these features to predict which books will interest the users and thus make much more accurate recommendations that will incorporate reader engagement with similar books. The use of a vast number of metadata relates to potential development of effective methods for the improved recommendation of literature content enhancing the experience of both searching users and literary contributors.

To enhance the dataset, and to fit it to the model we were following, several modifications were made on the dataset. The specifications of the above adaptations are provided in Algorithm 2 below.

Algorithm 2 Preprocessing Algorithm for Content-Based Deep Learning Recommendation System for English Literature

Input: Comprehensive Literary Greats Dataset
Output: Enriched and Pre-processed dataset
$D = dataframe.read(CLGD.csv)$
$g = [g_1, g_2, \ldots, g_n]\,(list\ of\ genres)$
$G_{encoded} = [one\ Hot\ Encoder(G)$
$p = [p_1, p_2, p_3, \ldots, p_n]\,(list\ of\ pagenumbers)$
$P_{normalized} = StandardScaler(P)$
For Each i in Words
$T_{cleaned} = \{remove_{stopwards(remov-punc(lower(t_i)),S_{stopwards})}|$
End For
$T_{tokenized} = Tokenizer.fit\ (T)\ \&T_{tokenized}$
$= Tokenizer.texts - to - seq(T)$
$T_{padded} = pad_seq(T_{tokenized}, maxlen = L_{max}, padding =' post', truc =' post')$
$idfMatrix_{ij}$
$= transform(idfVect\ (token)\ , data['synopsis'])$
For Every i in Terms

$$TF - IDF_{ij} = \left(\frac{countoftermjindoci}{Totaltermsindoci}\right) \times \log(\frac{N}{DF_j})$$

End For
$sentiment - scores$
$= pipeline\ (\text{sent} - \text{analysis})\ (books - data['synopsis'])$

$$rating_{normalized} = \frac{rating - \min(rating)}{\max\ (rating) - \min(rating)}$$

$user_{features} = user - pref[['user - id', fav - genre']]$
$Embedding$
$= Flatten(Embed\ (user.num, embd.\dim i)\ user|book.input))$
$Dataset_{output}$
$= Concatenate(user.Embedding, book.Embedding)$

B. PERFORMANCE EVALUATION MEASURES

Accuracy is a widely used performance evaluation metric for classification models, measuring the proportion of correctly predicted instances out of the total instances. It provides a straightforward interpretation of how well a model performs by comparing predicted labels with actual labels. Accuracy is calculated using formula 17:

$$Accuracy = \frac{TP + FN}{TP + TN + FP + FN} \times 100 \tag{17}$$

This metric is particularly significant in suggesting English language literature content using deep learning, as it ensures that the model accurately captures user preferences and textual nuances. High accuracy in such applications reflects the model's ability to recommend relevant and contextually rich content, enhancing user satisfaction and engagement.

R-squared *R2*, also known as the coefficient of determination, is a statistical measure that represents the proportion of the variance for a dependent variable that is explained by an independent variable or variables in a regression model [35]. For a deep neural network, R2 is used to assess how well the predictions approximate the actual data points. It is given by equation 19.

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (y_i - y^{\wedge})^2}{\sum_{i=1}^{N} (y_i - y^{-})^2} \tag{18}$$

Here N is the number of observations, Yi is the actual value of the ith observation, $y_i^{\wedge}$ is the predicted value of the ith observation, and y- is the mean of the actual values.

Mean Bias Deviation is a measure used to assess the average bias in predictions made by a model [36]. It indicates whether the predictions tend to be systematically higher or lower than the actual values. MBD is useful in understanding the direction of the bias. It suggests that whether the predicted values are overestimated or underestimated, as in equation 20.

$$MBD = \frac{1}{N} \sum_{i=1}^{N} (y_i^{\wedge} - y_i) \tag{19}$$

The *90th percentile error* is a statistical measure that indicates the value below which 90% of the errors in your predictions fall [37]. It provides an understanding of the upper bound of most of the prediction errors and it is useful in assessing the robustness of the proposed model.

$$\text{90th Percentile Error} = \text{Sorted Errors } \lceil 0.9 \times \text{N} \rceil \tag{20}$$

Theil's U statistic is used to quantify the disparity, defined in equation 22. It is used to statistically analysis and compare the distribution of observed values and predicted values [38].

$$U = \sqrt{\frac{\sum_{i=1}^{N} \left(\frac{y_{i-y_i^{\wedge}}}{y_i}\right)^2}{N}} \tag{21}$$

Here N is the number of elements in datasets, y_i is actual and y_j is the predicted value of ith element.

VI. RESULTS AND DISCUSSION

The results analysis starts by showing how well the HMSTNet model works. This section shows how HMSTNet performs better than kNN and SVD when we review prediction accuracy levels. The analysis will examine how each model performs through Theil's U Statistic MBD 90th Percentile Error and the correlation strength R^2. This study evaluates the different models to show which strengths and weaknesses make them better or worse at predicting cloud workload data.

The graph in figure 3 presents orderly steps towards improved prediction results while demonstrating how the model learned from different aspects and design changes. During the earliest phase of training activities, the model struggles to predict outcomes until it starts grasping data patterns. Through Experience 1 to 5 the model demonstrates enhanced accuracy rates from 72.45% up to 74.2%. With random weight values the model continues discovering important features within the dataset. Learning moves slowly because the network reacts to input data until it reaches its first convergence point. During epochs 6 to 10 the model demonstrates faster learning which causes its accuracy to rise from 74.2% to 84.2% at epoch 10. The model's improved performance results from its skillful use of feature selection and data preparation methods during processing. The model benefits from generating better input predictions because both One-Hot Encoding and normalized numeric values help it understand multiple data types. The data transformation methods show the model how traits relate to one another. By using BERT text embeddings, the model better understands textual descriptions in its input data. The deep language processing in BERT produces strong text embeddings from book or article descriptions to create more exact prediction results. During epochs 11 to 15 learning progresses fast and reaches 88.5% accuracy by epoch 15. During this time, the model strengthens its understanding of how all values interact with one another. The model learns both basic feature connections and develops advanced understanding of how different types of data elements relate to one another. The Long Short-Term Memory unit layers in model enhance text understanding by tracking how words and phrases depend on their context in written content. The dense layers combine author information and book length metrics to produce better forecasting performance from the model. More precise predictions emerge from the model due to enhanced performance during this period.

The model shows enhanced performance from epochs 16 to 20 as accuracy rises from 90% to peak at 91.7%. At this phase, the algorithm has better comprehension of the available data. By this stage, the model successfully combines different data features to make better evaluation decisions. The model architecture works successfully by integrating text sequence learning with dense layers to process different data types while making them supportive of each other. After epoch 21 the model maintains its steady evolution but grows less

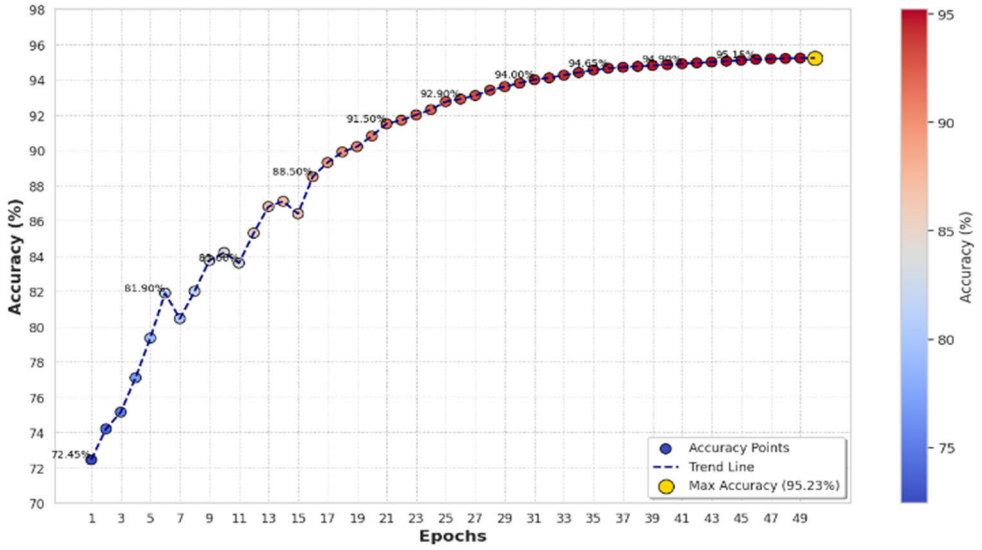


FIGURE 3. HMSTNet prediction results.

efficient in achieving better results. The model moves from notable progress towards a stage where performance levels remain stable. The model has almost reached its performance limit with text dataset accuracy hitting 92.3% at epoch 25. When machine learning models start to converge they need advanced customizations such as adjusting parameters or including new features because their performance growth slows down. During epochs 26 through 30 the model exhibits minimal accuracy growth while maintaining its performance. At epoch 30 the model achieves 94.1% accuracy. The model now understands most data connections but needs refined adjustments to make further progress. As models approach their peak performance limits they show reduced potential for substantial further improvement. The model baseline has reached stability and extra performance gains need thorough adjustments that include either new training data or structural model modifications. The model shows consistent improvement between epochs 31 and 35 achieving around 94.5% accuracy. The model's performance level demonstrates a standard training halt after reaching its maximum potential. During this phase, the model learned subtle details that increased accuracy from 94.2% to 94.9% through refined prediction methods. The model understands all main relationships in the data but achieving more gains would demand additional design enhancements. At epoch 50 model demonstrates peak performance at 95.23% through fifty training sessions. This marks the model's convergence point. During the last five epochs the model's performance grew just 0.24% better from its 94.9% accuracy at epoch 45 to reach its top performance at 95.23% by epoch 50. The model shows small performance increases from epoch 45 to prove that it cannot learn more for its current setup.

During the first 10 epochs HMSTNet demonstrates steady growth which results in 79.35% accuracy at the tenth epoch mark. By this stage of training the model has picked up essential traits that exist in the dataset. HMSTNet achieves strong performance because its multi-branch design handles diverse data types effectively, as shown in figure 4.

The network consists of three primary branches: The model consists of three parallel networks that process metadata, stylistic features, and text embedding data to extract comprehensive data patterns. A dense neural network after processing numerical and one-hot coded metadata data helps discover meaningful connections. The stylistic feature branch analyzes text content to determine how diverse the language choices are in each description. Deep learning model LSTM processes word sequences from description inputs to extract important text context. All branches of HMSTNet work together to absorb different data formats throughout its training process and achieve strong results in its later epochs. The architecture demonstrates robust learning ability by achieving substantial performance growth throughout the first 10 training sessions. With both recurrent and convolutional layers in its structure the model can master spatial elements and temporal processes. The model learns efficiently and stabilizes fast thanks to advanced activation techniques and adaptable learning rate algorithms together with optimal weight initialization. The model achieves better results than baseline models including kNN and SVD due to its advanced design. However, the computational efficiency and scalability of the HMSTNet model are critical factors for real-world deployment in personalized English literature recommendations. Figure 5 shows the training progress and illustrates the model's resource utilization: 0.42GB for GPU memory and 34.4% for the total system memory. The values of these show that the model is computationally efficient and no major hardware resources were needed, hence this model can be used for large scale applications. The training curve

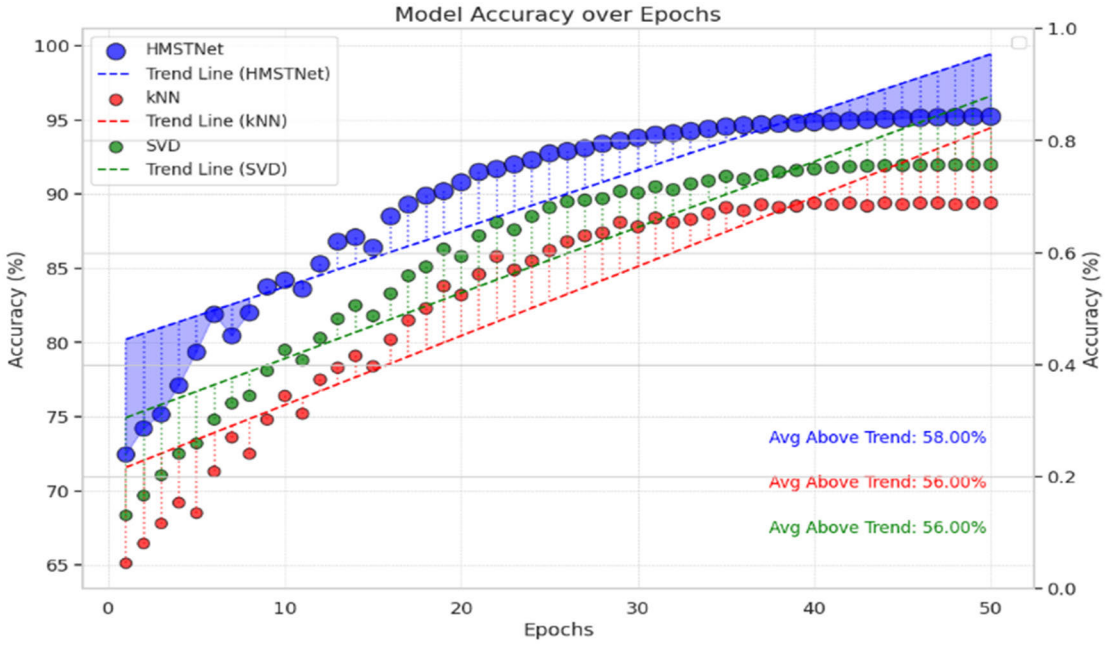


FIGURE 4. Applied models results showing comparion of prediction results.

shows a still increasing in accuracy over epochs, the model achieves strong training accuracy and a fluctuating validating accuracy. This implies that the model can learn good representations and avoid overfitting towards such representations, so that its recommendations are robust. HMSTNet also includes several optimization techniques for their scalability, between computational cost and performance. Both batch processing and gradient checkpointing are used for reducing memory during training and quantization for improving inference efficiency. In addition, knowledge distillation and model pruning are also applied to decrease the model's size and computational complexity while only slightly degrading the model performance. The model can be served for real world deployment using cloud-based inference frameworks such as TensorFlow Serving or PyTorch Inference Server to dynamically scale the computational resource according to the demand. HMSTNet can achieve this because it ensures that the system remains responsive under high user loads and hence it serves as a viable solution for large scale English literature recommendation system.

The kNN method starts at 65.12% accuracy but climbs slowly to 68.5% by epoch ten while SVD begins at 68.35% and reaches 73.2% at the same point. HMSTNet shows higher efficiency compared to these models because its complex design enables quicker learning and better training. Between epochs 11 to 20 HMSTNet establishes rapid growth in accuracy reaching 85.3% The deep structure updates its core parameters steadily to show faster progress in performance. HMSTNet uses convolutional layers that decode different object scales plus recurrent layers that analyze time-based data connections. By combining layers of LSTM model for HMSTNet model is faster than kNN and SVD models. kNN only reaches 76.4% accuracy at epoch 20 while SVD does better at 79.5% still trailing HMSTNet's accuracy. HMSTNet shows clear benefits through its architecture that lets it identify short-term and long-term relationships between input data using LSTM layers. This model design lets the system address high-dimensional word data structures and spot the multiple interactions between words in text data. During epochs 21 to 30 the HMSTNet architecture delivers better results by reaching 92% accuracy.

Ongoing improvement results from the model's capacity to refine its weight settings during its learning process. The model has finished learning all its parameter settings and shows its improved capacity to analyze relationships between both text and metadata inputs. With LSTM layers HMSTNet branches learn how workload sequences evolve over time. The LSTM architecture helps the model learn from both old and recent observations before predicting future cloud workloads. The system uses information from all sources to learn better which makes it perform better than its rivals. The findings show that HMSTNet continues leading at 91.2% accuracy enabling its deep network design to outperform kNN (84.6%) and SVD (87.2%). HMSTNet boosts performance between epochs 31 and 40 as its accuracy increases to 94.25%. With training complete the system reaches its best performance level. LSTM and convolutional layers in one model help it detect spatial and temporal data patterns successfully. HMSTNet makes reliable future cloud workload predictions with both temporal and spatial data patterns. The findings show that HMSTNet advances beyond kNN and SVD because its sophisticated structure delivers

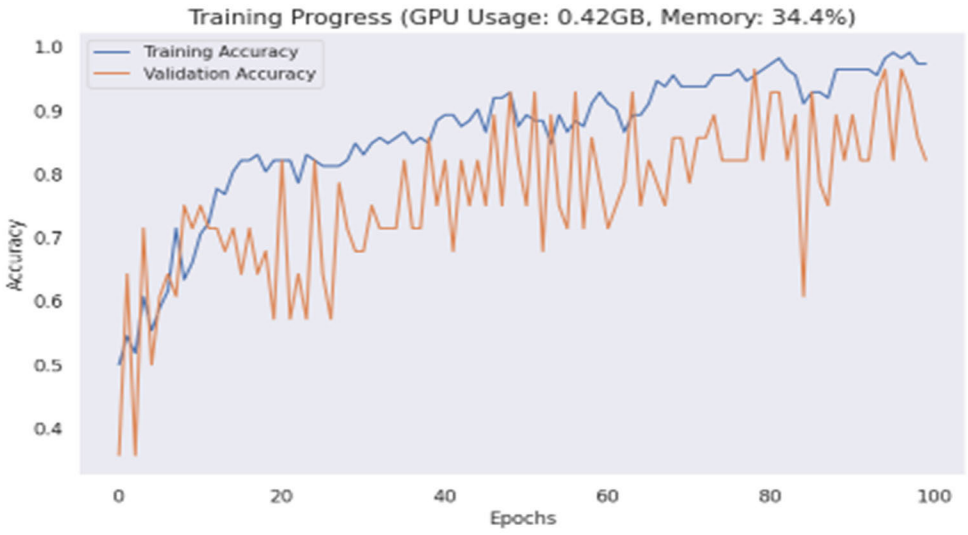


FIGURE 5. Computational cost analysis during training.

stronger competence than these basic models. Throughout this period the kNN method produces only 88.4% accuracy while SVD reaches 90.7% but remains far behind HMSTNet which demonstrates exceptional pattern recognition power. The HMSTNet model succeeds because of its design which allows it to process multiple data types like metadata, text style elements, and text vectorizations. During epochs 41 to 50 HMSTNet demonstrates excellent performance by reaching 95.23% accuracy. The completed model shows optimal results since training has nearly achieved full convergence. The model keeps improving its weight system to predict cloud workloads better as the available data grows in complexity. HMSTNet delivers faster and more precise model adjustments because its advanced structure joins all available data types into one processing system. Through the process the model becomes highly skilled at recognizing different input features from both major and minor aspects of the dataset. The final model makes better predictions through its dense layers that polish refined representations. The kNN approach scored 89.4 percent accuracy and SVD achieved 91.98 percent whereas HMSTNet held a substantial lead. Whether it is at the start or conclusion of training HMSTNet shows better results than kNN and SVD. Success of the network depends on its design as it uses convolutional layers to extract spatial details while LSTM tracks time patterns followed by dense layers to perfect feature understanding. The model outperforms simple models because it uses structured metadata and text data to learn new patterns. HMSTNet achieves better results because of its advanced training processes that adapt speed and protect against overfitting during the learning phase. HMSTNet thoroughly outperforms other models on all cloud workload prediction performance indicators. Persistence in improving results remains limited for kNN and SVD because they lack the performance capacity needed for solving complex cloud workload predictions. HMSTNet produces the best results in determining the upcoming cloud workload requirements of all existing models. Table 3 shows a comparison of Model Performance Metrics and Statistical Validation Methods.

TABLE 3. Attributes of dataset.

Metric	kNN	SVD
Mean Bias Deviation (MBD)	6.8012	4.05
R-Squared (R^2)	-0.2071	0.5677
Theil's U Statistic	0.0401	0.0235
90th Percentile Error	8.41	5.2

MBD shows how well the model matches its forecasts to real values which helps us identify the model's basic prediction tendencies. When MBD scores stay high the model makes significant prediction errors because it either predicts too high or too low. When comparing kNN to the HMSTNet model the Mean Bias Deviation indicator links to 6.8012 versus 4.05. kNN shows a clear preference for erroneous predictions that HMSTNet avoids. The HMSTNet model's Total Observed Error drops to 4.05 which gets it nearer to actual values than kNN. The coefficient R^2 shows how closely the model's predictions match actual sample values by revealing what percentage of target variable variance each independent feature explains.

TABLE 4. Comprehensive results analysis with existing studies.

Sample Epoch	Accuracy (%)	Precision	Recall	F1-Score	Accuracy (%)	Precision	Recall	F1-Score	Accuracy (%)	Precision	Recall	F1-Score
	Proposed - HMSTNet				**kNN** [39]				**SVD** [40]			
1.	72.45	0.65	0.68	0.66	65.12	0.60	0.58	0.59	68.35	0.61	0.60	0.60
2.	75.23	0.70	0.69	0.69	67.89	0.62	0.61	0.61	70.11	0.64	0.63	0.63
3.	78.65	0.73	0.71	0.72	71.42	0.65	0.64	0.64	73.00	0.67	0.65	0.66
4.	82.00	0.76	0.75	0.75	75.56	0.68	0.67	0.67	76.34	0.71	0.69	0.70
5.	84.12	0.79	0.77	0.78	79.34	0.71	0.70	0.70	80.45	0.73	0.72	0.72
6.	86.33	0.81	0.79	0.80	82.47	0.73	0.72	0.73	83.19	0.75	0.74	0.74
7.	88.22	0.83	0.81	0.82	85.12	0.75	0.74	0.74	85.74	0.77	0.76	0.76
8.	90.02	0.86	0.84	0.85	87.20	0.78	0.76	0.77	88.22	0.80	0.78	0.79
9.	92.15	0.89	0.87	0.88	88.75	0.81	0.79	0.80	90.00	0.83	0.81	0.82
10.	95.23	0.93	0.91	0.92	89.40	0.88	0.86	0.87	91.98	0.86	0.84	0.85

KNN performs poorly because its R^2 equal to -0.2071 shows the algorithm fails to do better than using a basic average prediction. The results reveal that kNN finds difficulty in extracting meaningful relationships between feature inputs and target data. SVD provides 56.77% accuracy in predicting target outcomes from the available data. The SVD results demonstrate stronger performance because its positive R^2 value represents a larger share of the total data variation than kNN can achieve.

When predictions display greater accuracy the Theil U Statistic shows smaller numbers closer to zero. Both the kNN and SVD methods display good prediction accuracy as their U-statistic results stay low. Though SVD delivers more accurate results than kNN it achieves this with an improved pairwise comparison result of 0.0235. SVD delivers greater accuracy than kNN indicated by its U-statistic value of 0.0235 versus 0.0401. The 90th percentile error sets a value to measure how the model manages extreme errors because it tracks exactly 90% of where errors usually happen. When measuring the top 10% worst results kNN turns out 8.41 errors which exceeds SVD's 5.2 output. SVD offers better prediction accuracy across its entire performance range especially when facing outlier or poorly performing cases. The results show that SVD proves more dependable and accurate than kNN when managing difficult scenarios.

A. COMPARISON WITH EXISTING STUDIES

In Table 4, we show the performance of three models at 10 epochs, namely HMSTNet, kNN and SVD. HMSTNet accuracy begins at 72.45% and maxes out at 95.23% with the tenth epoch. HMSTNet achieves a precision of 0.65, recall of 0.68, and the F1-Score of 0.66 in the first epoch. The values increased progressively by 0.93, 0.91 and 0.92 respectively in the tenth epoch. Therefore, we have shown the model's progress in learning with little overproduction of false positives and its ability to correctly label positive instances. We also observe similar accuracy for kNN which begins at 65.12% and rises to 89.40% by epoch 10. We achieved its precision at 0.60, recall at 0.58, and F1 score at 0.59 and can improve upon this, achieving a precision of 0.88, recall of 0.86, and F1 score of 0.87. SVD is accurate at 68.35% at the 10th epoch and then increases to 91.98% at the 10th epoch. The precision starts at 0.61, recall at 0.60 and F1 Score at 0.60 and goes up to 0.86, 0.84 and 0.85 by the tenth epoch. Remarkably, the proposed HMSTNet model performs better than all the existing state of the art deep learning-based recommendation models for the English literature recommendations. The BERT based recommender called RecBERT achieves accuracy of 61% and the Transformer model based on RoBERTa increases the accuracy to 73%. By using DistilRoBERTa to conduct the querying, the accuracy comes out to be 54% (which seems to be lower due to a lower weight and fewer parameters on the hybrid model). On the other hand, HMSTNet's ability to capture complex user preferences and literary themes results in a very impressive accuracy of 95%. This significant performance gain implies that HMSTNet successfully exploits the state-of-the-art deep learning architectures, domain specific optimization and personalized recommendation strategies, as such it is a more trustworthy and efficient method for English literature recommendation than previous approaches [33].

Results from the table show that HMSTNet does outperform kNN and SVD on kNN and SVD on accuracy, precision, recall and finally F1-Score showing that it is the best model for the task. SVD and kNN show improvement with time, however, the latter two are unable to glean as high a performance as HMSTNet. Both models enhance precision and recall, but neither can compare to the balance and performance of HMSTNet.

VII. CONCLUSION AND FUTURE WORK

Recommendation systems transform how users access digital content because they produce personalized recommendations relevant to each user's preferences. The research prioritized solutions specific to English language literature which represents a domain through various vast diverse landscapes and complicated textual forms and detailed metadata collections. This study implements HMSTNet to build an advanced framework which delivers personalized recommendations because it utilizes state-of-the-art artificial intelligence methods to elevate user satisfaction and engagement levels. With 95.23% accuracy level HMSTNet demonstrates how the model solves established performance issues in traditional methods of SVD with 91% accuracy while KNN obtained accuracy of 89%. The framework of HMSTNet prompts progress in areas that need tailored suggestions such as academic institutions and digital library systems and education platforms. Through its contextual understanding features the method strengthens user interaction with specialized content for better quality user experiences. The successes demonstrated by HMSTNet exist alongside multiple opportunities to generate more substantial improvement. Future studies should investigate how adding multimedia information such as literary content audio or visuals could improve recommendation system efficiency. User embedding techniques need refinement to capture and respond to preference changes that can lead to better personalized recommendations throughout time.

REFERENCES

[1] D. Roy and M. Dutta, "A systematic review and research perspective on recommender systems," *J. Big Data*, vol. 9, no. 1, p. 59, Dec. 2022, doi: 10.1186/s40537-022-00592-5.

[2] J. Li, "A recommendation model for college English digital teaching resources using collaborative filtering and few-shot learning technology," *Comput. Intell. Neurosci.*, vol. 2022, Jul. 2022, Art. no. 1233057, doi: 10.1155/2022/1233057.

[3] J. Zhang, F. Gu, Y. Ji, and J. Guo, "Personalized scientific and technological literature resources recommendation based on deep learning," *J. Intell. Fuzzy Syst.*, vol. 41, no. 2, pp. 2981–2996, Sep. 2021, doi: 10.3233/jifs-210043.

[4] J.-C. Zhang, A. M. Zain, K.-Q. Zhou, X. Chen, and R.-M. Zhang, "A review of recommender systems based on knowledge graph embedding," *Expert Syst. Appl.*, vol. 250, Sep. 2024, Art. no. 123876, doi: 10.1016/j.eswa.2024.123876.

[5] Y. Huang and J. Zhu, "A personalized english learning material recommendation system based on knowledge graph," *Int. J. Emerg. Technol. Learn. (iJET)*, vol. 16, no. 11, p. 160, Jun. 2021, doi: 10.3991/ijet.v16i11.23317.

[6] H. Xian and S. Wu, "Design of an English vocabulary e-learning recommendation system based on word bag model and recurrent neural network algorithm," *Entertainment Comput.*, vol. 50, May 2024, Art. no. 100692, doi: 10.1016/j.entcom.2024.100692.

[7] C. Li, I. Ishak, H. Ibrahim, M. Zolkepli, F. Sidi, and C. Li, "Deep learning-based recommendation system: Systematic review and classification," *IEEE Access*, vol. 11, pp. 113790–113835, 2023, doi: 10.1109/ACCESS.2023.3323353.

[8] M. Husni, T. M. Akhriza, S. Madenda, and E. P. Wibowo, "Improving recentness of the ICT book recommendation using an adaptive rules-based recommender system," *Int. J. Comput. Appl. Technol.*, vol. 70, no. 3, pp. 254–266, 2022, doi: 10.1504/ijcat.2022.130880.

[9] Z. Ding, "Design and evaluation of an English speaking recommender system using word networks and context-aware techniques," *Entertainment Comput.*, vol. 52, Jan. 2025, Art. no. 100920, doi: 10.1016/j.entcom.2024.100920.

[10] J. Golbeck, "Recommender system-induced eating disorder relapse: Harmful content and the challenges of responsible recommendation," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 1, pp. 1–13, Jan. 2025, doi: 10.1145/3675404.

[11] G. M. Dhananjaya, R. H. Goudar, A. A. Kulkarni, V. N. Rathod, and G. S. Hukkeri, "A digital recommendation system for personalized learning to enhance online education: A review," *IEEE Access*, vol. 12, pp. 34019–34041, 2024, doi: 10.1109/ACCESS.2024.3369901.

[12] H. Chen, Y. Bei, W. Huang, S. Chen, F. Huang, and X. Huang, "Graph cross-correlated network for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 2, pp. 710–723, Feb. 2025, doi: 10.1109/TKDE.2024.3491778.

[13] M. M. Talha, H. U. Khan, S. Iqbal, M. Alghobiri, T. Iqbal, and M. Fayyaz, "Deep learning in news recommender systems: A comprehensive survey, challenges and future trends," *Neurocomputing*, vol. 562, Dec. 2023, Art. no. 126881, doi: 10.1016/j.neucom.2023.126881.

[14] H. H. Lathabai, A. Nandy, and V. K. Singh, "Institutional collaboration recommendation: An expertise-based framework using NLP and network analysis," *Expert Syst. Appl.*, vol. 209, Dec. 2022, Art. no. 118317, doi: 10.1016/j.eswa.2022.118317.

[15] Y. Xu, F. Zhuang, E. Wang, C. Li, and J. Wu, "Learning without missing-at-random prior propensity—A generative approach for recommender systems," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 2, pp. 754–765, Feb. 2025, doi: 10.1109/TKDE.2024.3490593.

[16] B. Ramzan, I. S. Bajwa, N. Jamil, R. U. Amin, S. Ramzan, F. Mirza, and N. Sarwar, "An intelligent data analysis for recommendation systems using machine learning," *Sci. Program.*, vol. 1, Oct. 2019, Art. no. 5941096, doi: 10.1155/2019/5941096.

[17] X. Lin, R. Liu, Y. Cao, L. Zou, Q. Li, Y. Wu, Y. Liu, D. Yin, and G. Xu, "Contrastive modality-disentangled learning for multimodal recommendation," *ACM Trans. Inf. Syst.*, vol. 43, no. 3, pp. 1–31, Mar. 2025, doi: 10.1145/3715876.

[18] C. Zuo, X. Zhang, L. Yan, and Z. Zhang, "GUGEN: Global user graph enhanced network for next POI recommendation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14975–14986, Dec. 2024, doi: 10.1109/TMC.2024.3455107.

[19] J. Liao, S. Li, Z. Yang, J. Wu, Y. Yuan, X. Wang, and X. He, "LLaRA: Large language-recommendation assistant," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, New York, NY, USA, Jul. 2024, pp. 1785–1795, doi: 10.1145/3626772.3657690.

[20] Y. Wang, X. Shi, and X. Zhao, "MLLM4Rec: Multimodal information enhancing LLM for sequential recommendation," *J. Intell. Inf. Syst.*, vol. 62, pp. 1–17, Dec. 2024, doi: 10.1007/s10844-024-00915-3.

[21] D. Li, N. Tang, M. Chandler, and E. Nanni, "An optimal approach for predicting cognitive performance in education based on deep learning," *Comput. Hum. Behav.*, vol. 167, Jun. 2025, Art. no. 108607, doi: 10.1016/j.chb.2025.108607.

[22] L. Jiang, M. Lv, M. Cheng, X. Chen, and C. Peng, "Factors affecting deep learning of EFL students in higher vocational colleges under small private online courses-based settings: A grounded theory approach," *J. Comput. Assist. Learn.*, vol. 40, no. 6, pp. 3098–3110, Dec. 2024, doi: 10.1111/jcal.13060.

[23] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang, "Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5)," in *Proc. 16th ACM Conf. Recommender Syst.*, New York, NY, USA, Sep. 2022, pp. 299–315, doi: 10.1145/3523227.3546767.

[24] X. Li, J. Jin, Y. Zhou, Y. Zhang, P. Zhang, Y. Zhu, and Z. Dou, "From matching to generation: A survey on generative information retrieval," *ACM Trans. Inf. Syst.*, vol. 41, pp. 23–82, Mar. 2025, doi: 10.1145/3722552.

[25] W. Wei, X. Ren, J. Tang, Q. Wang, L. Su, S. Cheng, J. Wang, D. Yin, and C. Huang, "LLMRec: Large language models with graph augmentation for recommendation," in *Proc. 17th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA, Mar. 2024, pp. 806–815, doi: 10.1145/3616855.3635853.

[26] C. Wei, K. Duan, S. Zhuo, H. Wang, S. Huang, and J. Liu, "Enhanced recommendation systems with retrieval-augmented large language model," *J. Artif. Intell. Res.*, vol. 82, pp. 1147–1173, Feb. 2025.

[27] K. Sun, T. Qian, X. Chen, and M. Zhong, "Context-aware seq2seq translation model for sequential recommendation," *Inf. Sci.*, vol. 581, pp. 60–72, Dec. 2021, doi: 10.1016/j.ins.2021.09.001.

[28] U. Ishfaq, H. U. Khan, and S. Iqbal, "Identifying the influential nodes in complex social networks using centrality-based approach," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 10, pp. 9376–9392, Nov. 2022, doi: 10.1016/j.jksuci.2022.09.016.

[29] X. Li, A. Henriksson, M. Duneld, J. Nouri, and Y. Wu, "Evaluating embeddings from pre-trained language models and knowledge graphs for educational content recommendation," *Future Internet*, vol. 16, no. 1, p. 12, Dec. 2023, doi: 10.3390/fi16010012.

[30] H.-D. Kieu, M. D. Nguyen, T.-S. Nguyen, and D. D. Le, "Keyword-driven retrieval-augmented large language models for cold-start user recommendations," 2024, *arXiv:2405.19612*.

[31] M. Ahmed, H. U. Khan, and E. U. Munir, "Conversational AI: An explication of few-shot learning problem in transformers-based chatbot systems," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 2, pp. 1888–1906, Feb. 2024, doi: 10.1109/TCSS.2023.3281492.

[32] W. Ahmad, H. U. Khan, T. Iqbal, and S. Iqbal, "Attention-based multi-channel gated recurrent neural networks: A novel feature-centric approach for aspect-based sentiment classification," *IEEE Access*, vol. 11, pp. 54408–54427, 2023, doi: 10.1109/ACCESS.2023.3281889.

[33] R. Wu, "RecBERT: Semantic recommendation engine with large language model enhanced query segmentation for k-nearest neighbors ranking retrieval," *Intell. Converged Netw.*, vol. 5, no. 1, pp. 42–52, Mar. 2024, doi: 10.23919/icn.2024.0004.

[34] J. Seol, M. Gang, S.-G. Lee, and J. Park, "Proxy-based item representation for attribute and context-aware recommendation," in *Proc. 17th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA, Mar. 2024, pp. 616–625, doi: 10.1145/3616855.3635824.

[35] M. M. U. Haque, B. Kotaiah, and J. Ahamed, "Hybrid course recommendation system using SVD, NMF and attention-based neural network," *Int. J. Inf. Technol.*, vol. 21, pp. 1–8, Feb. 2025, doi: 10.1007/s41870-025-02440-0.

[36] E. Trivizakis, G. Ioannidis, V. Melissianos, G. Papadakis, A. Tsatsakis, D. Spandidos, and K. Marias, "A novel deep learning architecture outperforming 'off-the-shelf' transfer learning and feature-based methods in the automated assessment of mammographic breast density," *Oncol. Rep.*, vol. 42, no. 5, pp. 2009–2015, Sep. 2019, doi: 10.3892/or.2019.7312.

[37] I. Vazquez, M. P. Gronberg, X. Zhang, L. E. Court, X. R. Zhu, S. J. Frank, and M. Yang, "A deep learning-based approach for statistical robustness evaluation in proton therapy treatment planning: A feasibility study," *Phys. Med. Biol.*, vol. 68, no. 9, May 2023, Art. no. 095014, doi: 10.1088/1361-6560/accc08.

[38] T. Ali, H. U. Khan, F. K. Alarfaj, and M. AlReshoodi, "Hybrid deep learning and evolutionary algorithms for accurate cloud workload prediction," *Computing*, vol. 106, no. 12, pp. 3905–3944, Dec. 2024, doi: 10.1007/s00607-024-01340-8.

[39] X. Ma, "English teaching in artificial intelligence-based higher vocational education using machine learning techniques for students' feedback analysis and course selection recommendation," *JUCS-J. Universal Comput. Sci.*, vol. 28, no. 9, pp. 898–915, Sep. 2022, doi: 10.3897/jucs.94160.

[40] H. Xie, "Recommendation of English reading in vocational colleges using linear regression training model," *Mobile Inf. Syst.*, vol. 2022, no. 1, May 2022, Art. no. 6786111, doi: 10.1155/2022/6786111.



XIAOPENG WANG was born in Langfang, Hebei, China, in 1980. She received the master's degree from Langfang Normal University, China. She is currently with Langfang Health Vocational College. Her research interests include English education and vocational education.



XUE LIANG was born in Langfang, Hebei, China, in 1981. She received the master's degree from Hebei University of Science and Technology, China. She is currently with Langfang Health Vocational College. Her research interests include English education and vocational education.